

DAY 1 — Foundations

1. PLC Basics (15 min)

What it is: A PLC (Programmable Logic Controller) is an industrial computer designed to control machinery — motors, valves, conveyors — by continuously reading inputs (sensors, buttons) and driving outputs (lamps, motors, solenoids) based on a program.

Why it replaced relay panels: Before PLCs, industrial control was done with racks of physical relays wired together — every logic change meant re-wiring. A PLC replaces that wiring with software logic. Same electrical inputs/outputs, but the “logic” now lives in a program you can edit in minutes instead of a rewire job that takes hours. This is the single most important thing to land in this session — everything else follows from it.

Analogy to use: “Imagine your house wiring, but instead of physically rewiring a switch every time you want a light to behave differently, you just change a line of code.” Relays = hardware logic. PLC = software logic, same physical I/O.

Real-world use cases to mention: - Conveyor belts (start/stop, jam detection) - Packaging lines (fill, seal, count) - Elevators (floor sequencing, door interlocks) - Traffic lights (sequencing — ties directly into Day 2’s project)

Key terms to define here: Input, Output, Program/Logic, Scan (just the word — full explanation comes in block 3).

Common misconception to preempt: Students often think a PLC is “just a computer.” Clarify: it’s ruggedized for industrial environments (vibration, heat, electrical noise) and runs one program in a continuous, predictable loop — not a general-purpose OS multitasking arbitrary programs.

2. PLC Hardware (30 min)

CPU & Power Supply (8 min) The CPU is where the program actually executes — it reads inputs, solves the logic, writes outputs, every scan. The power supply converts incoming AC (110/230V) to the DC (24V typically) that powers the CPU and I/O modules. Point out: most industrial PLCs run on 24V DC for the I/O side specifically because it’s low enough to be safer to touch and standard across the industry.

Input Modules (8 min) Convert real-world signals (button presses, sensor states) into a form the CPU can read. Explain digital vs analog inputs here just at a naming level (analog gets its own full day). Digital input modules typically read 24V DC as “1” (true) and 0V as “0” (false, or NO/NC depending on wiring — the module doesn’t know your intent, it just reads voltage).

Output Modules (8 min) Three common output types, worth naming all three since students will encounter all of them on real jobs: - **Relay outputs** — mechanical, can switch AC or DC, slower, but robust and isolate the PLC electrically. - **Transistor outputs** — fast switching, DC only, no mechanical wear (good for high-frequency switching). - **Triac outputs** — AC only, solid-state, common for lamp/solenoid control.

Label exercise (6 min) Hand out a printed PLC diagram. Have students physically point to and label CPU, power supply, input terminal block, output terminal block. This is deliberately hands-on and low-stakes — it’s building spatial familiarity with the hardware before anything gets wired for real.

3. PLC Scan Cycle (15 min)

The core concept — spend real time making sure this lands: A PLC doesn’t run code line-by-line the way a general program does and stay there. It runs a continuous loop, typically completing in milliseconds:

1. **Input scan** — read the current state of every physical input and store it in memory (the input image table). Inputs are *not* re-read mid-logic — they’re frozen for the whole cycle.
2. **Logic solve** — execute the entire ladder program top to bottom, using the frozen input values from step 1, writing results to an internal output image table (not the physical outputs yet).
3. **Output update** — write the output image table to the actual physical output terminals, all at once.

Then it repeats. This is why “scan time” matters — a slow scan means slower response to real-world events.

Why this matters for debugging (say this explicitly): Every weird bug where “the output didn’t update when I expected” traces back to not understanding *when* inputs are actually read versus when logic uses them. If a button changes state mid-scan, the PLC won’t see it until the *next* scan cycle.

Whiteboard walkthrough: Draw one simple rung (e.g., a single NO contact driving a coil) and trace it through all three phases as a class, showing the input value getting “frozen,” used in logic, then written out.

4. Inputs/Outputs + NO/NC Contacts (40 min)

Wiring a push-button input (10 min) Physically wire a push button into an input terminal. Show the terminal numbering convention, common vs signal wire, and how to verify a connection with the PLC software’s I/O status view before assuming the wiring is correct.

Wiring a lamp output (10 min) Same process for an output — wire a lamp/LED indicator to an output terminal, confirm it’s on the correct output module type (see output types above).

Live status monitoring (10 min) Show the online status view — live green/red or highlighted indication of which inputs/outputs are currently active. This is the single most useful debugging tool students will use all week; make sure they’re comfortable navigating to it unprompted.

NO vs NC in real circuits (10 min) - NO (Normally Open): contact is open (no connection) at rest, closes when actuated. Logic reads “0” until pressed. - **NC (Normally Closed):** contact is closed at rest, opens when actuated. Logic reads “1” until pressed.

Why E-stops are always NC — this is the important safety concept to land clearly: If an E-stop is wired NC and someone cuts the wire, unplugs it, or it fails, the circuit reads as “pressed” (open) and the machine stops — a wiring fault fails safe. If it were wired NO instead, a cut wire would look identical to “not pressed,” and a real safety fault could go completely undetected. This is a genuine industrial safety standard, not just convention — worth stating plainly.

5. Ladder Logic Basics + Tag Naming (60 min)

Rungs & basic instructions (15 min) A “rung” is one horizontal line of logic in ladder diagram — conceptually similar to one line of code, but drawn as a circuit: contacts (conditions) on the left, coil (output) on the right. Power “flows” left to right if the conditions are true.

- **AND** — two NO contacts in series. Both must be true for the coil to energize.
- **OR** — two NO contacts in parallel (branches). Either being true energizes the coil.
- **NOT** — represented by using an NC contact instead of NO for that condition.

Guided practice (15 min) Build 2–3 simple rungs together as a class before releasing students to work independently — this is deliberately added time versus jumping straight to solo work, since ladder logic syntax is genuinely unfamiliar to first-timers and guided reps prevent a lot of frustrated debugging later.

Symbolic addressing & tag naming (15 min) Explain the difference between raw addressing (e.g., `I0.0`, `Q0.1` — the physical terminal address) and symbolic tags (e.g., `Start_PB`, `Motor_Run` — a human-readable name mapped to that address). Emphasize: professional programs are almost never written with raw addresses visible in the logic — they use tags, because six months later nobody remembers what `I0.3` was wired to. Establish a simple naming convention now (e.g., `PascalCase` or `snake_case`, verb/noun ordering) so it’s consistent for the rest of the week.

Commenting rungs (5 min) Show how to add a rung comment/description. Frame it as: “the rung shows you *what* the logic does; the comment tells you *why*.” A one-line comment explaining intent saves enormous debugging time for someone else (or future-you) reading the program later.

Project (10 min): Two-button AND logic (lamp on only if both pressed), then convert to OR logic (lamp on if either pressed) — using named tags throughout, not raw addresses.

6. Day Project — Motor Start/Stop with Seal-In Circuit (50 min)

The core concept students need to grasp: A momentary push button (Start) needs to turn on a motor that *stays* on after the button is released — this requires the output to “seal itself in” using its own contact as a self-holding condition, combined with a Stop button (NC, wired so pressing it breaks the circuit) that overrides the seal.

Logic pattern (teach this explicitly, it's the first "real" pattern of the week): (Start OR Motor_Run) AND NOT Stop → Motor_Run

The Motor_Run contact in parallel with Start is what creates the "seal-in" — once energized, the coil's own contact keeps the rung true even after the Start button is released.

Walkthrough structure: 1. Plan (5 min) — sketch the seal-in logic on paper before touching software. This forces them to reason about the pattern rather than trial-and-error clicking. 2. Wire I/O (10 min) — Start button (NO), Stop button (NC), motor output. 3. Write the rung (15 min) — implement the seal-in pattern from the sketch. 4. Test & debug (15 min) — common failure mode: forgetting the seal-in contact entirely (motor only runs while Start is held) — flag this as the most common mistake to watch for while circulating. 5. Class review (5 min) — surface the 2-3 most common mistakes seen and address them together.

If finished early: Add forward/reverse control with interlocking — two motor directions that can never both be active at once. This introduces mutual exclusion logic (each direction's rung includes a NOT of the other direction's output) — the same interlocking concept reused for safety circuits on Day 5.

DAY 2 — Timers, Counters & Troubleshooting

1. Timers: TON, TOF (30 min)

TON (Timer On-Delay) (12 min) Starts counting when its input condition goes true; after the preset time (PT) elapses, its output (Q) goes true. If the input goes false before the preset is reached, the timer resets to zero — it does *not* pause and resume. Key parameters to name explicitly: PT (preset time), ET (elapsed time, counts up while input is true), Q (output, true once $ET \geq PT$).

Analogy: A TON is like a kitchen timer that only counts if you're actively holding the start button — let go before it dings, and it resets to zero, not just pauses.

Demo: Wire a TON so a lamp blinks on after a fixed delay from a button press — students can watch ET climb live in the software and see Q flip once it hits PT .

TOF (Timer Off-Delay) (12 min) Opposite behavior: output (Q) goes true immediately when input goes true, but *stays* true for the preset time after the input goes false. Used for things that need to keep running briefly after the trigger stops.

Real-world example to use: A cooling fan that should keep running for 30 seconds after a motor shuts off, to dissipate heat, rather than stopping instantly.

Preset value experimentation (6 min) Let students change PT values live and observe the timing change — this cements the parameter's role better than any explanation.

Common student mistake: Confusing TON and TOF direction — walk through both diagrams side-by-side if confusion shows up, since the names alone (On-delay vs Off-delay) don't make the behavior obvious to a first-timer.

2. Counters (CTU, CTD) + Edge Detection (35 min)

CTU (Count Up) (8 min) Increments its current value (CV) by one each time its count input transitions from false to true (a rising edge — not while held true). Reaches a done state when $CV \geq \text{preset (} PV \text{)}$. Has a separate reset input to zero CV .

CTD (Count Down) (7 min) Same idea in reverse — decrements CV on each rising edge, useful for “count down remaining items” scenarios.

Edge-triggered instructions — R_TRIG / F_TRIG (10 min) This is a genuinely easy-to-miss but critical concept: a counter *should* only increment once per button press, not continuously while the button is held (since the button is “true” for the entire duration of a press, which might span multiple scan cycles). R_TRIG (rising edge trigger) outputs a single one-scan-long pulse the moment an input transitions from false to true. F_TRIG does the same on a false transition. Counters and many real applications use edge detection internally — worth showing explicitly rather than letting students assume it's automatic.

Concrete demonstration to run: Wire a counter *without* edge detection and hold a button down — watch the count value increment rapidly (multiple scans register the held button as multiple counts). Then add R_TRIG and repeat — one press, one count. This contrast is the fastest way to make the concept click.

Project (10 min): Count parts using edge-triggered logic on a simulated sensor input, reset the counter with a button.

3. Online Monitoring & Forcing I/O (25 min)

Watching live rung states (8 min) Show how energized rungs are typically highlighted (often green, or bolded) in the software's online/monitor mode. Students should get comfortable reading a live ladder diagram and immediately identifying which conditions are true/false without needing to guess.

Forcing inputs/outputs for testing (10 min) “Forcing” overrides the real physical I/O value with a value you set manually in software — lets you test logic without needing to physically trigger a sensor or button. Explain the real industrial use case: testing logic changes on a live system before the actual field device is wired, or diagnosing whether a fault is in the PLC logic or the physical wiring/sensor itself.

Important safety caveat to state explicitly: Forcing outputs on a real machine can be dangerous — a forced motor output will run the motor regardless of interlocks or sensor states. This is a simulator/training environment lesson; on a live industrial system, forcing has to be done with extreme caution and proper lockout procedures.

Reading fault/diagnostic indicators (7 min) Show common fault indicators (module fault LEDs, PLC run/stop/error states in software) and how to interpret them as the first troubleshooting step before diving into the logic itself.

Project: Force a sensor input high without triggering it physically, confirm the counter responds as if a real part passed; force an output on/off to test a lamp circuit independent of its normal trigger logic.

4. Combine Timer + Counter, Group Troubleshooting (25 min)

Retentive vs non-retentive timers (10 min) A standard TON resets to zero if interrupted. A *retentive* timer (sometimes RT0) keeps its accumulated value even if the input goes false, and only resets on an explicit reset signal — used when you need to track cumulative time across multiple separate trigger events (e.g., total run-time of a motor across several starts/stops, rather than resetting each time it stops).

Live debugging exercise (10 min) Deliberately plant 2–3 common mistakes into a pre-built program (e.g., missing edge detection causing a counter to over-count, a non-retentive timer used where a retentive one was needed) and have the class diagnose them together using the monitoring tools from block 3. This reinforces the troubleshooting skill in context rather than as an abstract lecture.

Project (5 min): Modify the counter from earlier so it also requires a 3-second delay before counting — forces combining a TON and a CTU in a single rung, which is genuinely one of the more common real patterns (debounce-style delay before accepting a count).

5. Day Project — Traffic Light Controller (95 min)

The goal: Build a 3-light sequence (Red → Green → Yellow → Red, looping) using TON timers to control the duration of each phase.

Core teaching point — sequencing logic: This is the first project requiring *state*, not just simple reactive logic. Introduce the idea of a “step” or “state” variable (an internal memory bit or word) that tracks which phase of the sequence is currently active, with each TON timer gated by that state, and each timer’s completion advancing to the next state.

Structure to guide students through: 1. Design (10 min) — sketch a state diagram: three states, red timer duration, green timer duration, yellow timer duration, looping back to red. 2. Build (35 min) — implement each state as its own rung group: a state bit, a timer gated by that bit, and logic that advances to the next state when the timer’s Q goes true. 3. Test & debug (35 min) — most common bug: forgetting to reset the *current* state’s timer when transitioning away from it, causing incorrect timing on the next cycle through the loop. Watch for this specifically while circulating. 4. Class walkthrough (15 min) — have 1–2 students present their sequencing approach; there are usually multiple valid ways to structure the state logic, and seeing alternatives helps everyone.

If finished early: Add a pedestrian crossing button that interrupts the normal cycle and forces an all-red + walk phase — reuses the same state/timer pattern but introduces an interrupt condition, a preview of priority/override logic they’ll need in later projects.

DAY 3 — Analog & Data

1. Analog Inputs/Outputs (40 min)

How analog signals work (8 min) Unlike digital I/O (simple on/off), analog signals represent a continuous range of values — typically industry-standard **4-20mA** current loops or **0-10V** voltage signals. Explain why 4-20mA is so common industrially: because a “0” reading is ambiguous (is the sensor at zero, or is the wire broken/disconnected?), starting the range at 4mA means a reading of 0mA is unambiguously a fault, not a valid zero value. This is a genuinely useful real-world fact worth stating explicitly.

Raw ADC counts vs engineering units (8 min) The PLC’s analog input module doesn’t read “4-20mA” directly into a meaningful number — it converts the signal into a raw digital value via an ADC (Analog-to-Digital Converter), often a range like 0-1023 (10-bit) or 0-27648 (common in some PLC brands), which has no inherent physical meaning until it’s scaled.

Scaling formula walkthrough (12 min) This is the hardest concept of the week for most beginners — go slowly. The core idea is linear interpolation between two known ranges:

$$\text{Engineering Value} = ((\text{Raw} - \text{RawMin}) / (\text{RawMax} - \text{RawMin})) \times (\text{EUMax} - \text{EUMin}) + \text{EUMin}$$

Concrete example to work through on the board: raw range 0-1023 representing 0-100°C. - Raw = 512 → $((512-0)/(1023-0)) \times (100-0) + 0 \approx 50.05^\circ\text{C}$

Most PLC software provides a built-in scaling instruction (commonly `SCALE_X` or `NORM_X + SCALE_X` combined — `NORM_X` normalizes raw to a 0.0-1.0 float, `SCALE_X` maps that to the target engineering range) so students don’t have to write the raw formula by hand, but understanding the underlying math is what prevents blind trust in a black-box instruction.

Project (12 min): Take a simulated analog input (0-1023 raw), scale it to 0-100°C, display the result on screen.

2. Arithmetic Instructions (20 min)

ADD/SUB, MUL/DIV (14 min) Standard arithmetic blocks operating on the scaled register values from the exercise above — deliberately continuing the same data rather than introducing a fresh unrelated example, so it feels like a natural next step rather than a new topic.

Watch for this specific bug: Division by zero — if a divisor register can legitimately be zero (e.g., averaging over a count that could be zero), the program needs an explicit guard condition before the DIV instruction executes, or it will fault. Worth mentioning now since it becomes relevant again in more complex Day 4 logic.

Project (6 min): Compute the average of two analog readings (ADD the two scaled values, DIV by 2).

3. Comparison Instructions (20 min)

GT/LT/EQ concept (8 min) Boolean-output instructions that compare two values and output true/false — the bridge between “data” (a scaled temperature number) and “logic” (an alarm output). Emphasize: this is the pattern that turns raw analog data into an actionable digital decision, which is the whole point of scaling values in the first place.

Project (12 min): Trigger an output if scaled temperature exceeds 50°C — directly chains the scaling (block 1) and comparison (this block) together into one working pipeline.

4. Data Types (20 min)

BOOL vs INT vs REAL (10 min) - `BOOL` — true/false, one bit. Used for digital I/O and logic states. - `INT` — whole numbers, used for raw ADC counts, counter values. - `REAL` — floating-point (decimal) numbers, needed anywhere fractional precision matters — which is exactly why the scaled temperature value from block 1 needs to be REAL, not INT (50.05°C would truncate to 50 as an INT, losing precision).

Brief mention of arrays (5 min) An array is a numbered list of values of the same type under one tag name (e.g., storing readings from 4 sensors as `Temp[0]` through `Temp[3]` instead of four separate tags). Just a concept-level mention here — full use comes with more advanced programs beyond this course’s scope.

Why REAL matters (5 min) Directly tie back to the scaling exercise: ask students what would happen if the scaled temperature tag were declared as INT instead of REAL, and walk through the truncation/precision loss concretely.

5. Day Project — Temperature Monitoring System (110 min)

The goal: Chain everything from today into one working pipeline: analog sensor → scale to °C → compare against a setpoint → trigger a fan or alarm output, plus log an alarm event when the setpoint is crossed.

Structure to guide students through: 1. Design (10 min) — sketch the full data flow on paper: raw input → scaling block → REAL tag → comparison block → BOOL alarm output. Seeing it as a pipeline before building helps students avoid getting lost mid-build. 2. Build scaling + arithmetic (30 min) — implement the scaling from block 1, using a REAL tag for the output. 3. Add comparison + alarm/event log (30 min) — implement the setpoint comparison, and introduce a simple alarm/event log entry (a timestamped record written when the alarm condition first becomes true) — this is a light preview of the SCADA alarm concept that resurfaces properly on Day 5. 4. Test & debug (30 min) — common bug to watch for: comparing a REAL value using an exact `EQ` instead of `GT / LT` (floating-point exact-equality comparisons are unreliable due to precision — worth calling out explicitly if it comes up). 5. Review edge cases (10 min) — discuss what happens at exactly the setpoint boundary, and briefly what “hysteresis” means (a small dead-band around the setpoint to prevent the alarm chattering on and off if the reading hovers right at the threshold) — good to mention conceptually even without implementing it.

If finished early: Add a second setpoint band (low-temp alarm in addition to high-temp), or implement basic signal filtering (a moving average over the last 3–5 scan values) to smooth out noisy analog readings — genuinely useful since real sensors are rarely as clean as the simulated input.

DAY 4 — Structured Logic

1. Shift Registers (30 min)

Concept (10 min) A shift register tracks a sequence of items moving through discrete stations by “shifting” bit or word values along an array each scan (or each trigger event) — modeling something physically moving down a line without needing a separate sensor-tracking scheme for every single station.

Concrete framing to use: Imagine 4 stations on a conveyor. Each time a new item enters station 1, every existing tracked item “shifts” one position down (station 1’s status moves to station 2’s slot, etc.), and station 1 resets to reflect the new item. This mirrors physically how items move on a real line.

Live demo (10 min) Simulate a 4-station line, manually trigger a “new item” input, and watch the register values shift down the array in the software’s live view — visually this is one of the more satisfying concepts to demo since students can directly see data “moving.”

Project (10 min): Simulate 4 stations, “push” a bottle through and track its position with each scan.

2. Program Control — Subroutines & Function Blocks (40 min)

Why structured code (10 min) Everything so far has been flat rungs in one continuous program — fine for small projects, but doesn’t scale. A **Function Block (FB)** is a reusable, self-contained piece of logic with its own inputs/outputs, that can be called multiple times with different data (each call gets its own “instance” of internal memory). Contrast with a **Function (FC)**, which is stateless — no memory between calls, purely input-to-output.

When to use which — worth stating explicitly, it’s a common interview question: - Use an **FB** when the logic needs to remember something between scans (a timer, a counter, a state) — most real control logic.
- Use an **FC** for pure calculations with no memory needed (e.g., a unit conversion).

FB syntax and instance data (10 min) Explain “instance data block” — each time an FB is called, it gets its own dedicated memory block holding its internal state, separate from every other call to the same FB. This is what makes the same piece of logic reusable for, say, two independent sensors without them interfering with each other’s timers/counters.

Project (20 min): Convert Day 3’s temperature monitor into a callable FB (inputs: raw analog value, setpoint; outputs: scaled temperature, alarm bit), then call it twice for two independent sensors — demonstrating that each instance tracks its own state correctly and independently.

3. PID Basics (30 min)

Concept only — full tuning is out of scope this week, be upfront about that (15 min) PID stands for Proportional-Integral-Derivative — a control algorithm that continuously adjusts an output to minimize the difference (**error**) between a desired **setpoint** and the current measured value (**process variable**).

- **P (Proportional)** — output response scales directly with the current error. Bigger error, bigger correction.
- **I (Integral)** — accounts for accumulated past error over time, eliminating small persistent offsets a P-only controller can’t fully correct.
- **D (Derivative)** — responds to the *rate of change* of error, dampening overshoot by anticipating where the value is heading.

Simple analogy to use: Driving a car to maintain a speed limit — P is “how hard you press the pedal based on how far off you are right now,” I is “you’ve been slightly under for a while, so nudge up a bit more to compensate,” D is “you’re approaching the limit fast, so ease off before you overshoot.”

Simulated demo (15 min) Show a PID loop responding live to a setpoint change (e.g., simulated temperature control), pointing out the characteristic curve — initial correction, possible slight overshoot, settling at the setpoint. No hands-on tuning required this week; the goal is recognition, not mastery. Mention that real PID tuning (adjusting the P/I/D gain values to get a fast, stable response without excessive overshoot) is a legitimate multi-day topic on its own, and point interested students toward it as a “next step” resource.

4. Day Project — Batch Mixing Station (110 min)

The goal: A 3-stage sequential process — fill (timed) → mix (motor + timed) → drain (level-sensor-based) — built as separate function blocks called from one main sequencer, tying together everything from today (FBs) and the sequencing concept from Day 2's traffic light project.

Structure to guide students through: 1. Design (10 min) — sketch the sequence as a state diagram: Fill → Mix → Drain → Idle/Complete, noting what condition advances each state (timer done for fill/mix, level sensor for drain). 2. Build Fill FB (25 min) — a timer-based fill stage, packaged as its own function block. 3. Build Mix FB (25 min) — motor + timer, same FB pattern. 4. Build Drain FB (25 min) — level-sensor-triggered, introduces a non-timer-based state transition (worth pointing out explicitly, since every previous state transition this week has been timer-driven). 5. Integrate & test (25 min) — call all three FBs from one main sequencer program, verify the full fill→mix→drain cycle runs correctly end to end. Common bug to watch for: forgetting to reset a completed stage's internal state before the next cycle starts, causing the second run of the sequence to behave differently from the first.

If finished early: Add a manual/auto mode switch — in auto, the FB sequence runs automatically; in manual, an operator can directly jog the mixer motor. This introduces mode-selection logic (an interlock between automatic sequencing and manual override) — a pattern used constantly in real industrial programs, and a natural lead-in to the safety interlocking on Day 5.

DAY 5 — Networking, SCADA, Safety & Capstone

1. PLC Networking (15 min)

How PLCs talk to each other and to SCADA/HMI (10 min) A single PLC controlling one machine is common, but real facilities usually have multiple PLCs, HMIs (Human-Machine Interfaces — operator touchscreens), and SCADA (Supervisory Control and Data Acquisition — plant-wide monitoring/control software) all communicating over a shared industrial network. The PLC exposes its internal tag data over the network; HMI/SCADA systems read and write to those tags to display status and issue commands.

Real-world network topology example (5 min) Describe a simple example: several PLCs on a factory floor, each controlling one machine/line, all connected via Ethernet to a central SCADA server in a control room, with HMI touchscreens at each machine for local operator control. This is the “big picture” context that makes today’s SCADA block make sense — students have only worked with one isolated PLC all week until now.

2. Communication Protocols (15 min)

Modbus (5 min) One of the oldest and simplest industrial protocols — reads/writes numbered registers over serial (RTU) or Ethernet (TCP). Still extremely widely used because it’s simple and vendor-agnostic.

Profinet (5 min) Siemens-originated, Ethernet-based, designed for real-time industrial communication with tighter timing guarantees than plain Modbus TCP — common in TIA Portal / Siemens-heavy environments.

EtherNet/IP (5 min) Rockwell/Allen-Bradley-originated, also Ethernet-based, built on the same underlying CIP (Common Industrial Protocol) used across several other industrial networks.

Framing for students: They don’t need configuration depth on any of these today — the goal is recognition: if they hear these terms on the job, they should immediately know what category of thing is being discussed and roughly what it’s for.

3. Safety Basics — E-Stop & Safety Relay Wiring (25 min)

Dual-channel E-stop wiring (10 min) A single-channel E-stop (one NC contact, as covered Day 1) is a reasonable default for hobby/training work, but real industrial safety circuits typically use **dual-channel** wiring — two independent NC contacts feeding two separate monitoring channels, so a single failure in one channel doesn’t silently disable the safety function. This is worth explicitly connecting back to Day 1’s NC discussion: this is the same fail-safe principle, taken one level further for redundancy.

Safety relay basics (8 min) A dedicated safety relay module (separate from the PLC’s general I/O) monitors the dual-channel E-stop circuit and has its own certified, hardwired logic to cut power to the machine. **Key point to state clearly:** safety-critical stop functions are generally wired through dedicated hardware, not solely through PLC program logic — because a software bug or PLC fault shouldn’t be able to defeat the emergency stop. The PLC can *read* the safety relay’s status (to know the machine is stopped) but shouldn’t be the sole thing standing between “E-stop pressed” and “power actually cut.”

Project (7 min): Wire a dual-channel E-stop into the existing motor circuit from Day 1, and verify — by disconnecting one channel — that the fault is detected rather than silently ignored.

4. SCADA Integration (40 min)

HMI/SCADA tag binding concept (10 min) Explain tag binding: an HMI screen element (a lamp graphic, a numeric display, a button) is linked directly to a PLC tag. When the tag’s value changes in the PLC, the HMI display updates automatically — no separate communication logic needs to be written for it, it’s a configuration-level link, not new programming.

Project (30 min): Bind 2–3 tags from Day 4’s batch mixing FB to an HMI screen — a status lamp (mixing in progress), a level readout (numeric display), and Day 3’s alarm indicator. **Why this project matters as a moment, worth calling out to students:** this is the first time all week they see PLC-internal data rendered somewhere *outside* the PLC programming software itself — genuinely often a real “aha” moment connecting the whole week’s work into something that looks like an actual industrial operator screen.

5. Capstone Project — Automated Sorting/Packaging Line (90 min)

The goal: Teams combine essentially everything from the week — motor control, sensors, timers, counters, one analog input, the E-stop safety circuit, and SCADA monitoring — into one working system, with minimal step-by-step guidance from the instructor.

Structure to guide teams through: 1. Team planning (10 min) — assign roles (wiring, logic, HMI), sketch the overall system on paper before building anything. Encourage teams to explicitly list which Day 1–4 concept each part of the system will reuse — this is a good forcing function for them to realize how much they’ve actually learned. 2. Build motor control + sensors + timers/counters (25 min) — the core sorting mechanism. 3. Add analog input + E-stop safety circuit (25 min) — an analog sensor feeding a sort decision (e.g., sort by simulated size/weight reading), plus the dual-channel E-stop from block 3 wired into the whole system. 4. Add SCADA monitoring, integrate & test (30 min) — bind key tags to an HMI view and run the full line end-to-end.

Your role during this block: Deliberately step back from leading — circulate, answer questions, but let teams struggle through integration issues themselves. This is the one project of the week explicitly designed to be less guided, since the goal is testing whether the pieces have actually connected into working understanding rather than memorized steps.

If a team finishes early: Add a reject/second-sort branch (sort into 3 categories instead of 2 based on the analog reading) — extends the same logic pattern without needing any new instructions, a good use of remaining time before demos rather than idle waiting.

6. Team Demos + Wrap-up (25 min)

Live team demos (15 min) Each team runs their line live for the group. Keep it brief per team — the value here is seeing different valid approaches to the same problem, not an exhaustive review of every rung.

Q&A + where to go next (10 min) Close with a short, honest note on what wasn’t covered this week and where to go deeper: full TIA Portal or Codesys feature depth, Structured Text (a text-based PLC programming language, an alternative to ladder logic worth knowing exists), and OPC UA (a modern, more secure industrial communication standard increasingly replacing older protocols like Modbus in new installations). Frame this as “here’s your map for continuing,” not a discouraging list of gaps.

Note on Scope

OPC UA was left out of hands-on teaching entirely this week due to time — it’s mentioned only during Friday’s wrap-up as a “where to go next” pointer, not taught. **Safety PLCs / E-stop circuit design** is covered as a fixed Day 5 block (Section 3 above) rather than left optional, since it was judged more immediately useful than deeper protocol configuration for students heading toward shop-floor work.